Vehicle Number Plate Detection and Reading

Submitted by: **Harsh Verma** (as25dxb024)

AI in Operations · Day 1, Session III lab task (slide 69): *"Build a Python script that detects and reads vehicle number plates from an image."*

1. The goal

We built a program that looks at a photo of a vehicle, **finds the number plate**, and **reads the text on it** (for example **51F73420**). This is the system used at parking gates, toll booths and car parks, the same "parking lot" scenario shown on the slide.

2. How the system works (two steps)

Step	What it does	Tool used
1. Find the plate	Draws a box around every number plate in the photo.	**YOLO11n**, a convolutional neural network (CNN), re-trained by us on the labelled photos.
2. Read the plate	Cuts out the plate (a "crop") and turns it into text, with a confidence score.	**fast-plate-ocr**, a ready-made reader trained only on licence plates from 65+ countries and regions.

The plate finder is built from the same blocks as the MNIST digit lab in class (convolution and pooling layers). The plate reader also starts with convolution layers, then predicts one character at each position, much like MNIST predicted one digit.

3. The data

We used a free public dataset (**Vehicle Registration Plates**, Roboflow, CC BY 4.0) with **8,823 photos**, each with the plate marked by a person. Most photos come from a **Vietnamese parking-garage barrier camera** (cars and motorbikes), plus street photos from other countries. It is split into training (6,176), validation (1,765) and test (882) photos.

4. What we did, step by step

- **Checked the data first.** We found that some test photos were edited copies (rotated, brightened, cropped or bordered) of training photos. Testing on these would be like giving students the exam answers in advance, so we also report results on a **clean test set** (831 photos) without them.
- **Set up the training of the plate finder**, starting from a model that already knows everyday objects and teaching it to find plates (*transfer learning*). We switched off mirror-image flipping, because a mirrored plate never appears on a real camera.
- **Measured reading accuracy honestly.** The dataset has no plate text, so we read 30 clear plates by eye. A second, independent reviewer read them again, and both agreed on all 30.
- **Built one function,** `read_plates(image)`**,** that runs the whole system and marks unsure reads as **"needs review"** so that a person checks them.
- **Showed an operations use:** a parking-gate log that opens the barrier automatically for confident reads and sends doubtful ones to staff.
- **Exported a standalone Python script,** `anpr.py`**,** as the slide asked, and tested that it runs.

5. Results so far

Plate reading: 28 out of 30 plates read exactly right (**93%**); only **0.8%** of characters were wrong. Because the test is small, the true accuracy is likely between 79% and 98%. **Plate finding:** every part of the notebook was run start to finish without errors on a laptop, but only as a short test (1 training round on 5% of the photos), because a laptop is too slow for full training. So the plate-finding scores in that saved run are not meaningful yet. The full training (30 rounds on all photos) runs in Google Colab on a free GPU (estimated 35–45 minutes) and produces the final detection scores.

6. Limits and ethics

- Most photos are Vietnamese plates, so accuracy on other countries (for example UAE plates) still needs to be tested with local photos.
- The 30-plate reading test is small. A real deployment would test several hundred plates.
- Number plates are personal data. A real system should store logs only as long as needed, limit who can search them, and never fine anyone based on an unsure read without a human checking it.

Files: `ANPR_Number_Plate_Recognition.ipynb` (full notebook, runs in Google Colab) and `anpr.py` (command-line script).
Tools: Python, Ultralytics YOLO, fast-plate-ocr, OpenCV.